

今月のフロントエンド

フロントエンド エンジニア集会 2026 年 8 月

"今月のフロントエンド" とは

今月あったフロントエンドのニュースを, 以下のフォーマットで紹介します.

- ニュースがあった技術について, その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何があったかを解説

取り上げないもの

- 特定フレームワークに関するバージョンアップ (例外あり)
- AI 単品のニュース

Oxlint

2026/07/22

"Oxlint" とは

- Rust 製の JavaScript / TypeScript 向け Linter (コードの書き方を検査するツール)
- 定番の ESLint と同じ役割を, 桁違いの速さで実行することを目指している
- Vite を開発する VoidZero (2026 年 6 月に Cloudflare が買収) が開発している

Oxlint のニュース

型情報を使った Lint が正式版に — ESLint 比で最大 18 倍

- 7/22, 型を見る Lint (type-aware) が実験版から **安定版** へ
- typescript-eslint が持つ型ありルール **61 個のうち 59 個** をカバー
- VS Code 規模のコードベースで **ESLint + typescript-eslint の最大 18 倍** 速い
- 設定は `typeAware: true` の 1 行

なぜこれが大きいのか

- Lint には **書き方だけ見るもの** と **型情報まで見るもの** の 2 種類がある
 - 「Promise の `await` 忘れ」のようなバグは, 後者でないと見つけられない
- 型ありルールは重く, ESLint では CI で数分かかることも珍しくなかった
- Oxlint は速さが売りだったが, **型ありルールが無い** ことが最大の弱点だった
 - 「速いが物足りない」から「置き換え候補」へ

TypeScript 7 との関係

- 型を見る部分は **tsgolint** という別エンジンが担当している
- tsgolint は 7/8 に正式リリースされた **Go 製の TypeScript 7 コンパイラの上に直接構築**
(TypeScript 7.0 = コンパイラを Go へ書き直し, 型チェックを約 10 倍高速化した版)
- Lint と型チェックが **同じ型情報を共有** するため, 二重に解析しなくて済む
- 一方 ESLint 側は, 型情報を取り出すための API が TypeScript 7 でまだ未提供
→ **型ありルールは TypeScript 6 系で動かす** 必要がある状態が続いている

"Biome" とは

- 同じく Rust 製の, Linter と Formatter (整形ツール) を兼ねたツール
- ESLint (検査) と Prettier (整形) を **まとめて 1 つで置き換える** ことを目指す
- 2025 年に v2 へ. Oxlint と並ぶ「脱 ESLint」の有力候補として比較されることが多い

Biome と Oxlint, どちらが速いのか

同じ Rust 製なのに, 速度差は 2~5 倍つく

- oxc 公式のベンチマーク (VS Code のコードベース) では **Oxlint が 2.5~3.3 倍**
- 実プロダクト (TypeScript / TSX 約 800 ファイル) での計測では **約 4.9 倍**
- ただし **どちらも数秒以内** で終わる — ESLint の数十秒とは別の土俵にいる

なぜ差がつくのか — 構文木の作りが違う

	Oxlint (oxc)	Biome
構文木	AST (抽象構文木)	CST (具象構文木)
空白・コメント	保持しない	すべて保持する
メモリの確保	一括で確保	ノード単位に確保

- Biome は **整形もする** ため, 元のソースを 1 文字も失わない木が必要になる
- 構文エラーのある壊れたコードも整形・検査できるのは, この作りの利点

速さは「割り切り」の結果

- Oxlint の木は **検査に必要な情報だけ** — 木が小さく, 走査も速い
- メモリを一括確保 (アリーナ) することで **メモリへのアクセスが一
直線** になる
 - oxc はこの変更だけで約 20% 速くなったと報告している
- つまり速度差は最適化の巧拙ではなく, **担う役割の差** から来ている
 - 「Biome が遅い」ではなく「Biome は整形もやっている」

型情報の扱いも、思想が違う

	Oxlint	Biome
型の取得	TypeScript 7 に任せる	自前で推論
TypeScript	必要	不要
精度	tsc と同じ	一部のみ

- Biome は「TypeScript 無しで型ありチェック」という別解に賭けている
→ `await` 忘れの検出精度は typescript-eslint の 75~85% 程度
- Oxlint は逆に、型の解釈を **本家に任せて速く回す** 方針

で, どちらを選べばいいのか

- **速さとルールの網羅性** なら Oxlint
→ ESLint 由来のルールの移植が多く, 乗り換えで失うものが少ない
- **1 つで完結する体験** なら Biome
→ 整形も検査も設定ファイル 1 つ. JSON / CSS など JS 以外のファイルも扱える
- どちらも ESLint とは桁が違う速さ — **まず ESLint から離れること**の方が本題

Google Chrome

2026/07/28



"Google Chrome" とは

- Google が開発する, デスクトップで圧倒的シェアを持つ Web ブラウザ
- レンダリングエンジンは Blink, 新しい CSS / Web API の実装を積極的に主導
- Edge など他の Chromium 系ブラウザの基盤にもなっている

Google Chrome のニュース

Chrome 151 — SPA の "画面遷移" がようやく計測できるようになった

- `soft-navigation` : JavaScript による **画面内の遷移** を 1 回の遷移として記録
- `interaction-contentful-paint` : 操作後に **新しい内容が描画されるまで** を計測
- `<usermedia>` : カメラ / マイクの利用許可を **HTML の要素として** 置ける新タグ
- Web Crypto に **耐量子計算機暗号** (ML-KEM / ML-DSA) が試験的に追加

出典: [New in Chrome 151 \(Chrome for Developers\)](#).

何が嬉しいのか

- 従来の計測 API は **ページを読み込み直す遷移** しか見えていなかった
- React / Next.js のような SPA (画面をページ再読み込みなしで切り替える作り) では,
1 セッションで 5 回画面を移動しても **計測されるのは最初の 1 回だけ**
→ 残り 4 回は "存在しない遷移" として扱われていた
- Core Web Vitals (Google が定める体感速度の指標) を **画面ごとに** 出せるようになる
→ 「どの画面が遅いのか」が, ようやく数字で分かる

他ブラウザとの互換性

機能	Chrome / Edge	Safari	Firefox
<code>soft-navigation</code> 等の計測	151～	—	—
<code><usermedia></code>	151～	—	—
<code>shadowrootslotassignment</code>	151～	27 (先行)	151～

- 計測系は **非対応でも表示は変わらない** → 段階的強化で使える
- `<usermedia>` は許可 UI そのもの → 非対応ブラウザには **JS API の併用が必須**

来月から Chrome は 2 週間ごとになる

- 9/8 の **Chrome 153** から, 安定版のリリース間隔が **4 週間 → 2 週間** へ
- 対象はデスクトップ / Android / iOS のすべて. 年間 約 26 バージョンになる
- 1 回の変更量が減るため, 不具合の切り分けがしやすくなるとしている
- 企業向けの Extended Stable は 8 週間のまま

Cloudflare

2026/08/06

"Cloudflare" とは

- 世界中の Web サイトが利用する CDN (配信を高速化する仕組み) / セキュリティ事業者
- 利用者に近いサーバでコードを動かす実行基盤 **Workers** を提供している
- 2026 年 6 月に Vite の開発元 VoidZero を買収し, フロントエンド基盤側にも進出した

Cloudflare のニュース

"Kitesurf" — Chromium を使わずに作られたブラウザが登場

- 8/6 公開. **Chromium** を一切使わず, Workers の V8 隔離環境だけで動くブラウザ
- HTML の処理には Rust 製の **Blitz**, CSS エンジンには Firefox の **Styleo** を採用
- **Chrome DevTools Protocol 互換** — Puppeteer / Playwright がそのまま繋がる
- スクリーンショット 1 枚あたり CPU **1,173ms → 380ms**, メモリ **271MiB → 58MiB**

なぜフロントエンドの話なのか

- 想定用途は **AI エージェント** だが, 中身は「画面を持たないブラウザ」
→ E2E テストや OGP 画像生成で使う **ヘッドレスブラウザの置き換え候補** になる
- この用途は実質 Chromium 一択で, 1 プロセスあたりの資源消費が重いのが定番の悩みだった
- ただし **実時間 (待ち時間) は Chromium の約 1.8 倍遅い** — 省資源とのトレードオフ
- ベータ期間は無料で試せる. 将来的な OSS 化も表明されている

ブラウザエンジンを作り直す, という流れ

- ブラウザの中核 (エンジン) は Blink / WebKit / Gecko の 3 系統に集約されて久しい
- 一方で近年, **Rust で部品ごとに作り直す** 試みが実用水準に育ってきている
 - Blitz : レイアウトと描画を担う Rust 製エンジン
 - Stylo : Firefox が実際に使っている CSS エンジン. 単体で再利用できる
- Kitesurf は, その部品を組み合わせで **第 4 のエンジンを実際に商用投入した 例**

JavaScript

2026/08/13



"JavaScript" とは

- Web ブラウザ上で動作する, フロントエンド開発の中心となるプログラミング言語
- 言語仕様は ECMAScript として標準化され, 毎年 1 回改訂される
- 実装はブラウザごとに異なり, Chrome は V8, Firefox は SpiderMonkey, Safari は JavaScriptCore を使う

JavaScript のニュース

`using` 構文が, 3 大エンジンすべてで動くようになった

- 8/13 公開の **Safari 先行版 (Safari Technology Preview 250)** が Explicit Resource Management (明示的なリソース管理) に対応
- `using` / `await using`, `Symbol.dispose`, `DisposableStack` を含む完全対応
- これで **V8 / SpiderMonkey / JavaScriptCore** の 3 エンジンが出揃った
- TypeScript で書いて変換していた構文が, **変換なしで動く** 段階に入る

"using" とは何か

- ブロックを抜けるときに **後片付けを自動で呼ぶ** 変数宣言
 - ファイル, DB 接続, ロックなど「開いたら必ず閉じる」対象が想定用途
- `try { ... } finally { close() }` の書き忘れを, 構文レベルで防げる
- 複数ある場合は **宣言と逆順** に片付けられる. 非同期版が `await using`
- Java の try-with-resources, C# の `using`, Python の `with` に相当する機能

使えるようになるのはいつか

- Safari は **先行版での対応** — 製品版 (通常の Safari) への搭載はこれから
- Chrome / Firefox は既に安定版で利用可能
- 当面は TypeScript による変換を併用するのが安全
→ 「変換なしで書ける」と言い切れるのは, Safari 製品版に載ってから

おわり